Практическая Визуализация Данных на Python

Автор: Эшвин Паджанкар
Тип файла: pdf
Размер: 3,9 МБ
Язык: Английский
Страниц: 160

Практическая визуализация данных на Python: ускоренный подход к изучению визуализации данных с помощью Python

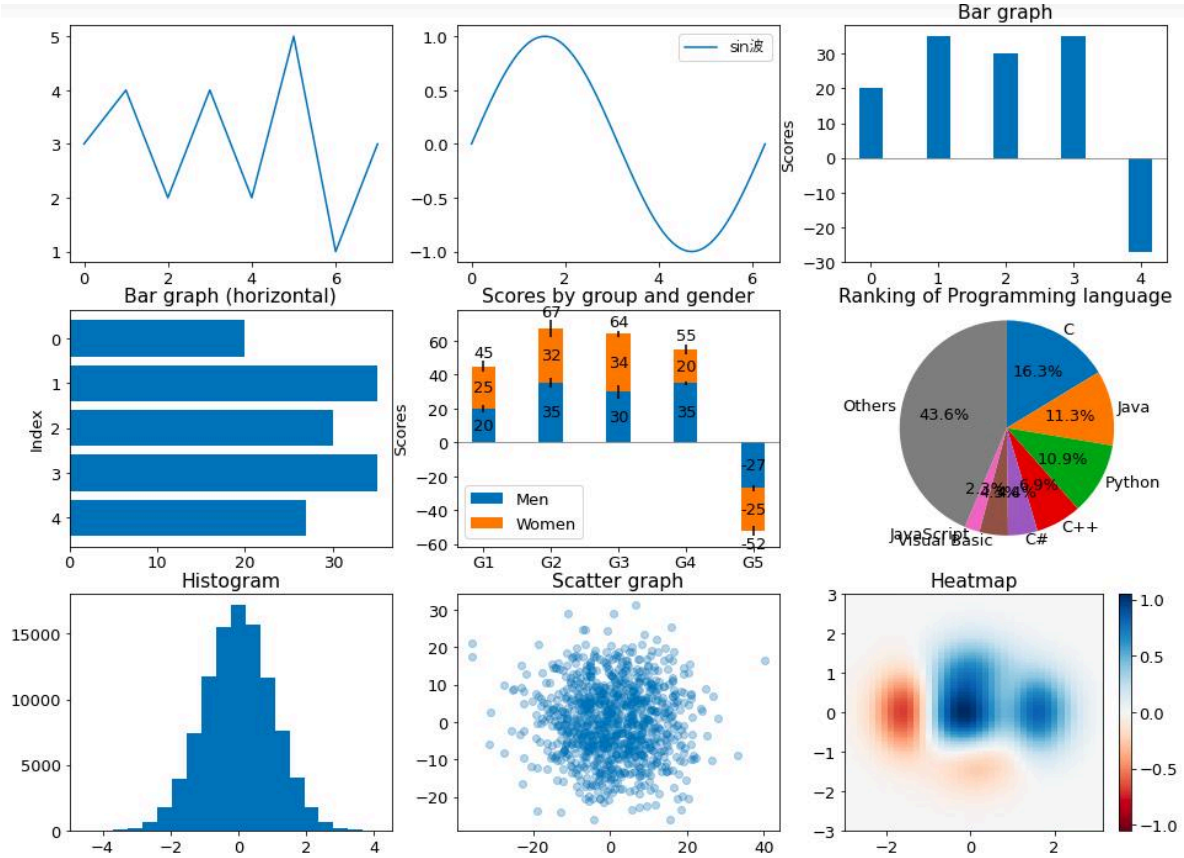
Введение

Визуализация данных — один из важнейших навыков для инженеров, аналитиков, исследователей и специалистов по работе с данными. Современные инженерные проекты генерируют огромные объемы информации: показания датчиков, результаты испытаний, результаты моделирования, производственную статистику, данные о потреблении энергии, финансовые показатели и журналы эксплуатации.

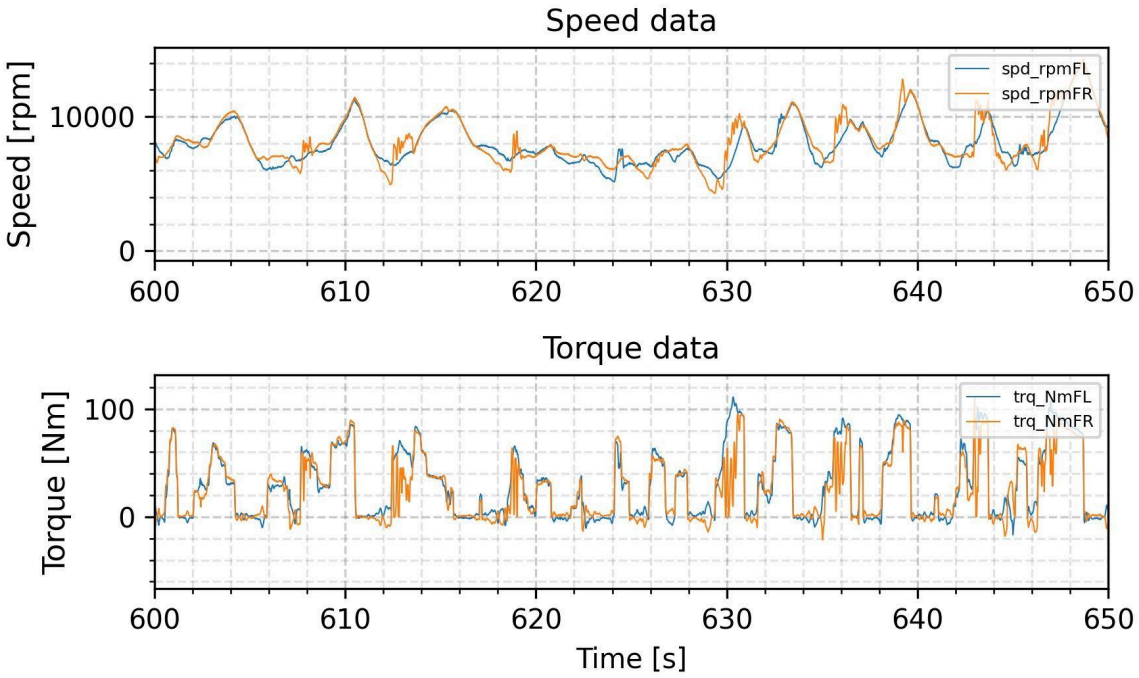
Просмотр тысяч числовых значений в электронной таблице редко позволяет эффективно понять, что происходит.

Python предлагает мощное решение. С помощью таких библиотек, как **Matplotlib, Seaborn, Pandas и Plotly**, инженеры могут преобразовывать необработанные числовые данные в наглядные визуальные представления 📊.

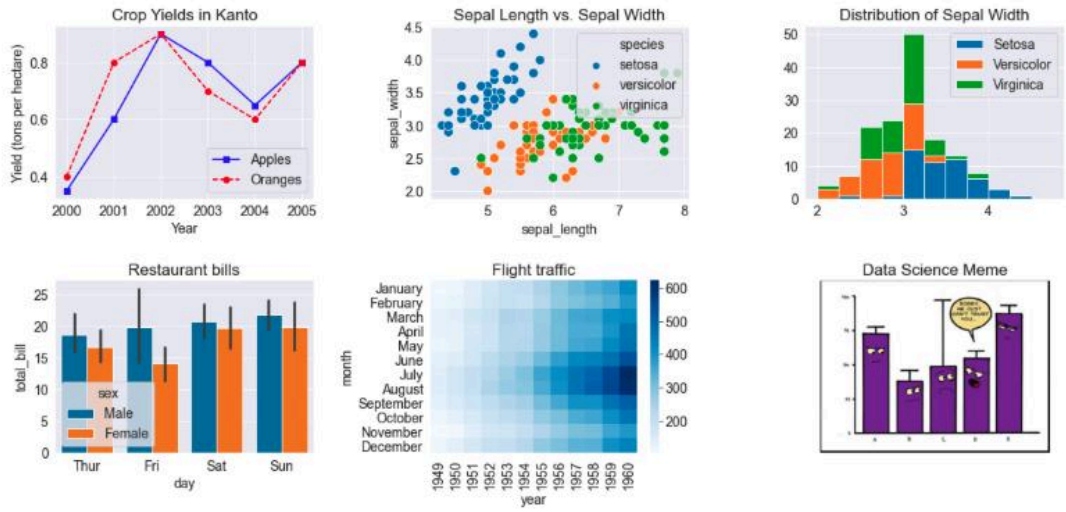
Хорошая визуализация может выявить тенденцию, которая почти незаметна в таблице, указать на аномальные показатели, продемонстрировать взаимосвязь между переменными или объяснить сложный инженерный результат неспециалисту.

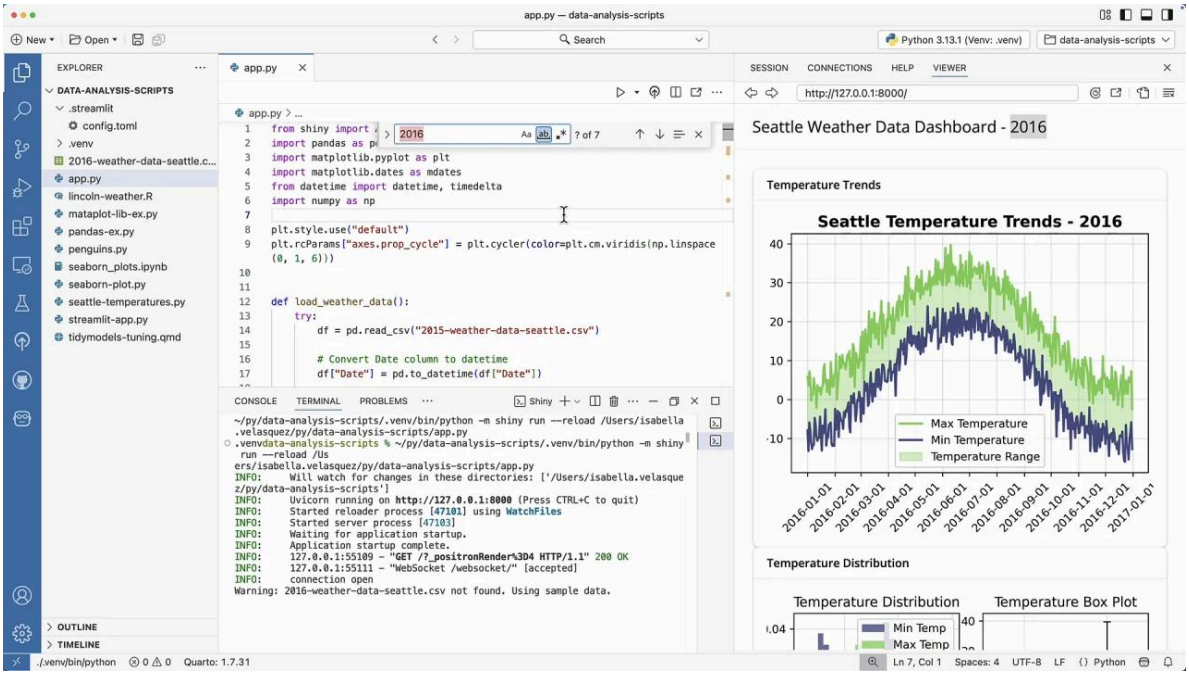


Python figure



Data Visualization using Python, Matplotlib and Seaborn





Цель этого ускоренного подхода не в том, чтобы просто научить вас создавать привлекательные графики. Настоящая цель — понять, **почему следует использовать именно эту визуализацию, как ее эффективно построить и как интерпретировать инженерную информацию, которую она передает**.

Независимо от того, являетесь ли вы студентом-механиком, анализирующим данные о температуре, инженером-строителем, изучающим измерения конструкций, инженером-электриком, анализирующим сигналы напряжения, или профессионалом, создающим аналитическую информационную панель, визуализация на Python может стать неотъемлемой частью вашего повседневного рабочего процесса.

Теоретическая база

Почему инженерам нужна визуализация данных

Инженерия в своей основе опирается на данные.

Рассмотрим систему мониторинга конструкций, которая измеряет:

- Напряжение
- Деформация
- Смещение
- Температура
- Вибрация
- Нагрузка

- Ускорение

Набор данных может содержать сотни тысяч наблюдений.

Таблица может выглядеть так:

Время (ы)	Нагрузка (кН)	Смещение (мм)	Напряжение (МПа)
0	0	0.00	0
10	20	0.12	42
20	40	0.27	85
30	60	0.44	126
40	80	0.63	170

Числовая информация полезна, но график сразу показывает, является ли зависимость линейной, нелинейной или на нее влияют аномальные измерения.

Принцип визуального кодирования

Визуализация преобразует данные в визуальные свойства, такие как:

- Положение
- Длина
- Форма
- Размер
- Цвет
- Ориентация

Например, на линейном графике переменная отображается по вертикали, а время обычно отображается по горизонтали.

С математической точки зрения, если инженерное измерение представлено

[y=f(x)]

то визуализация создает графическое представление зависимости между (x) и (y).

Таким образом, инженерная задача заключается не просто в построении графика зависимости между (x) и (y). Она заключается в выборе визуального представления, которое точно передает эту зависимость.

Определение

Что такое визуализация данных в Python?

Визуализация данных в Python — это процесс использования библиотек программирования Python для преобразования структурированных или числовых данных в графические представления, которые упрощают понимание закономерностей, взаимосвязей, распределений, тенденций и аномалий.

К наиболее часто используемым инструментам визуализации в Python относятся:

Library	Main Strength	Typical Engineering Use
Matplotlib	Flexible plotting	Scientific and engineering graphs
Seaborn	Statistical visualization	Data exploration and distributions
Pandas	Quick plotting	Fast exploratory analysis
Plotly	Interactive graphics	Dashboards and interactive reports
NumPy	Numerical processing	Preparing mathematical datasets

Visualization Is More Than Decoration

An engineering graph should answer a question.

For example:

Question: Does increasing temperature increase motor current?

A suitable visualization might plot:

$$I \quad \text{vs.} \quad T$$

where:

- I = motor current
- T = temperature

A visualization becomes valuable when the viewer can quickly understand the engineering relationship.

Step-by-Step Python Visualization Workflow

Step 1: Install the Required Libraries

A basic environment can be prepared using:

```
pip install pandas matplotlib seaborn plotly
```

Then import the libraries:

```
📊 import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
```

Step 2: Create or Load Data

For example:

```
import pandas as pd

data = {
    "Time": [0, 10, 20, 30, 40, 50],
    "Temperature": [22, 25, 29, 34, 38, 42],
    "Pressure": [101, 103, 106, 110, 114, 119]
}

df = pd.DataFrame(data)

print(df)
```

Now Python has converted the information into a structured DataFrame.

Step 3: Create a Basic Line Chart

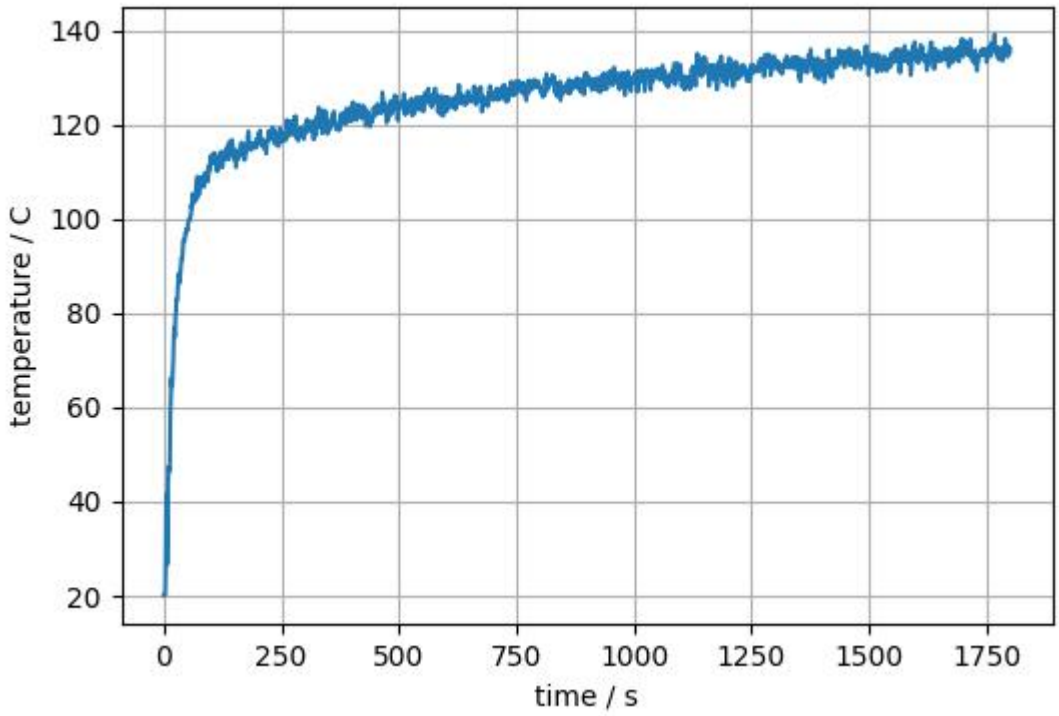
```
import matplotlib.pyplot as plt

plt.plot(df["Time"], df["Temperature"])

plt.xlabel("Time (s)")
plt.ylabel("Temperature (°C)")
```

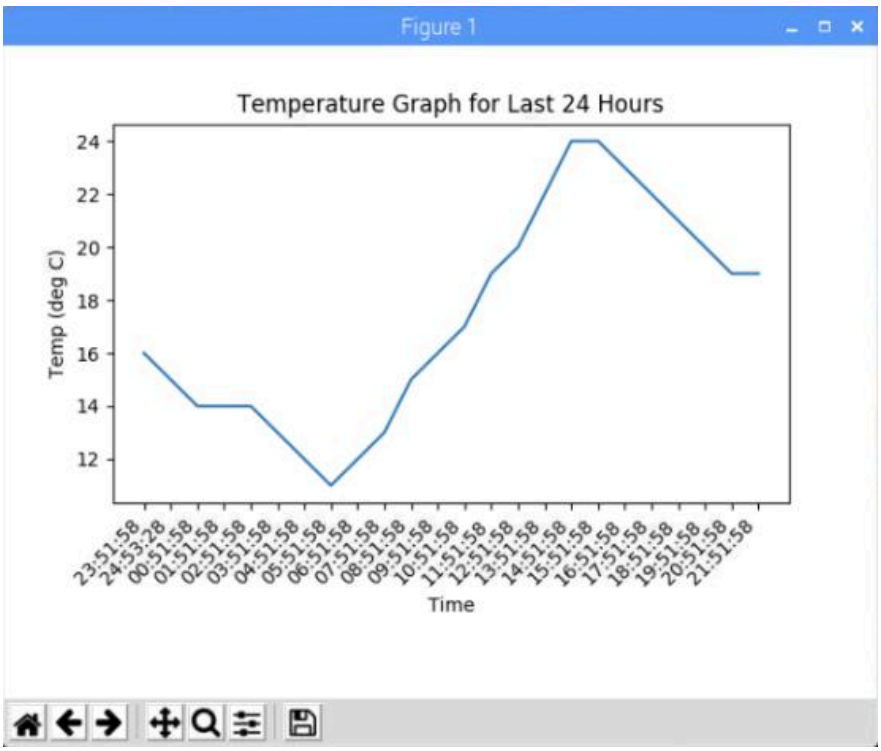
```
plt.title("Temperature Variation with Time")

plt.show()
```

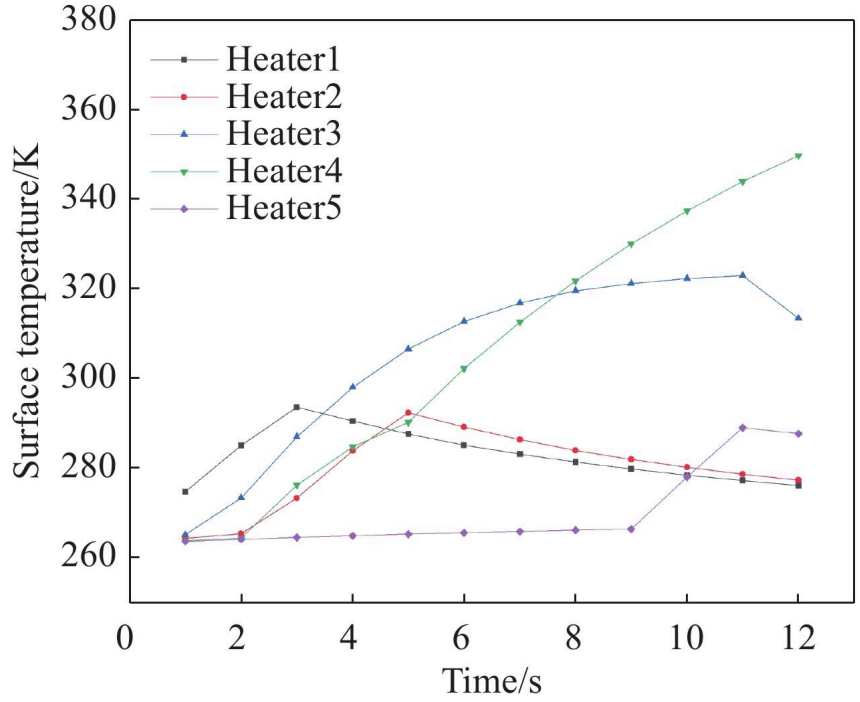
The basic relationship is:

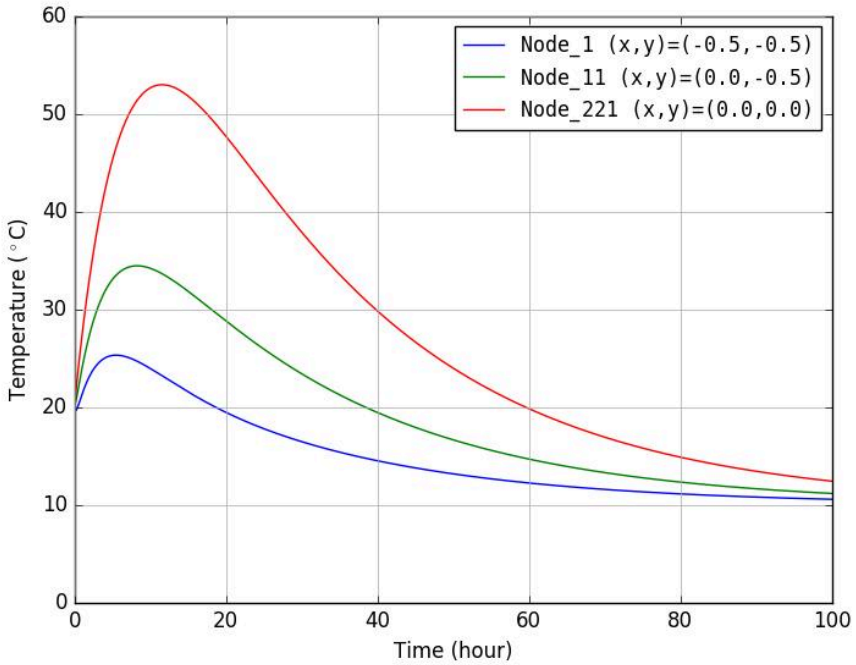
[T=T(t)]

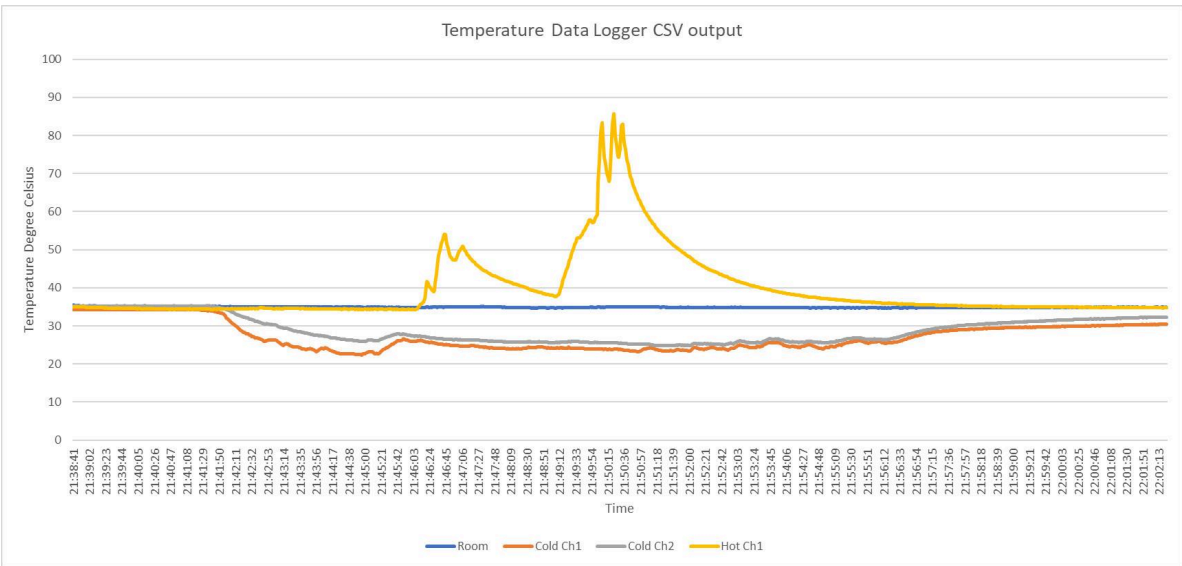
where temperature changes as a function of time.

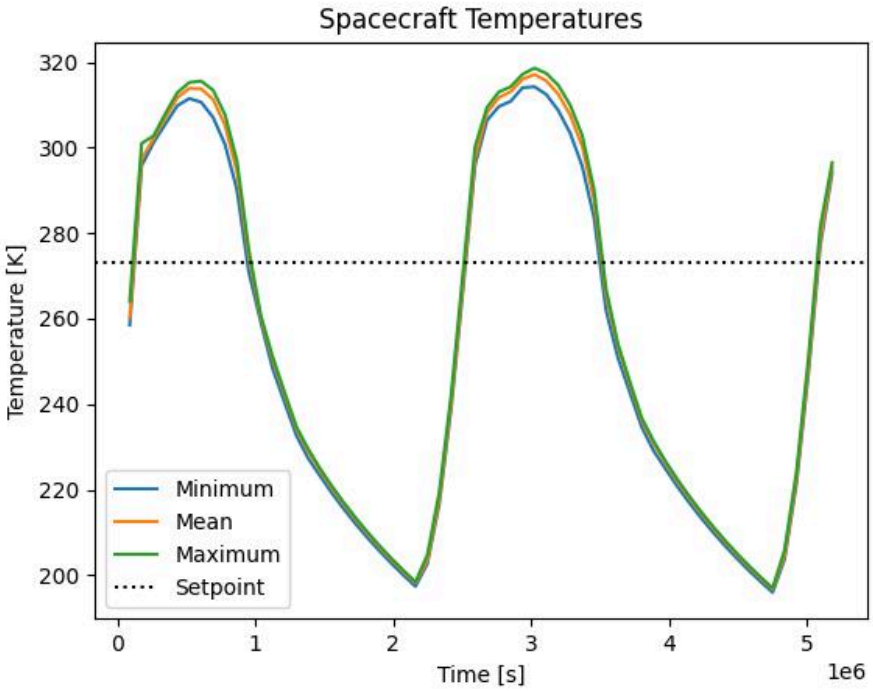





Heater1
Heater2
Heater3
Heater4
Heater5
Surface temperature/K
Time/s
380
360
340
320
300
280
260
0
2
4
6
8
10
12


Node_1 (x,y)=(-0.5,-0.5)
Node_11 (x,y)=(0.0,-0.5)
Node_221 (x,y)=(0.0,0.0)
Temperature (° C)
Time (hour)
60
50
40
30
20
10
0
0
20
40
60
80
100


Temperature Data Logger CSV output
Temperature Degree Celsius
Time
Room
Cold Ch1
Cold Ch2
Hot Ch1



Step 4: Improve the Engineering Chart

A professional visualization should include meaningful labels.

```
plt.figure(figsize=(9, 5))

plt.plot(
    df["Time"],
    df["Temperature"],
    marker="o",
    linewidth=2
)

plt.xlabel("Time (s)")
plt.ylabel("Temperature (°C)")
plt.title("Temperature Response During the Test")

plt.grid(True)
plt.tight_layout()
plt.show()
```

This small improvement makes the chart substantially easier to interpret.

Step 5: Add a Second Variable

Engineers frequently need to compare two measurements.

```
plt.figure(figsize=(9, 5))

plt.plot(df["Time"], df["Temperature"], marker="o",
         label="Temperature")

plt.plot(df["Time"], df["Pressure"], marker="s",
         label="Pressure")

plt.xlabel("Time (s)")
plt.ylabel("Measurement")
plt.title("Temperature and Pressure During Testing")

plt.legend()
plt.grid(True)
plt.show()
```

However, combining variables with very different units can sometimes produce a misleading visualization. In such situations, separate plots or carefully designed axes may be preferable.

Comparison: Choosing the Right Chart

Line Chart vs Bar Chart vs Scatter Plot

Choosing the correct chart is one of the most important visualization decisions.

Chart	Best For	Example
Line	Trends over continuous variables	Temperature vs time
Bar	Comparing categories	Energy consumption by building
Scatter	Relationships between variables	Stress vs strain
Histogram	Distribution	Measurement errors
Box plot	Statistical spread	Sensor performance
Heatmap	Matrix relationships	Correlation analysis
Pie chart	Simple proportions	Limited categorical data

Chart	Best For	Example
3D plot	Spatial/multivariable relationships	Surface geometry

When to Use a Scatter Plot

Suppose an engineer wants to investigate whether increasing load causes increasing displacement.

```
plt.scatter(df["Pressure"], df["Temperature"])

plt.xlabel("Pressure")
plt.ylabel("Temperature")
plt.title("Temperature vs Pressure")

plt.show()
```

A scatter plot is particularly useful when individual observations matter.

When to Use a Histogram

```
plt.hist(df["Temperature"], bins=5)

plt.xlabel("Temperature (°C)")
plt.ylabel("Frequency")
plt.title("Temperature Distribution")

plt.show()
```

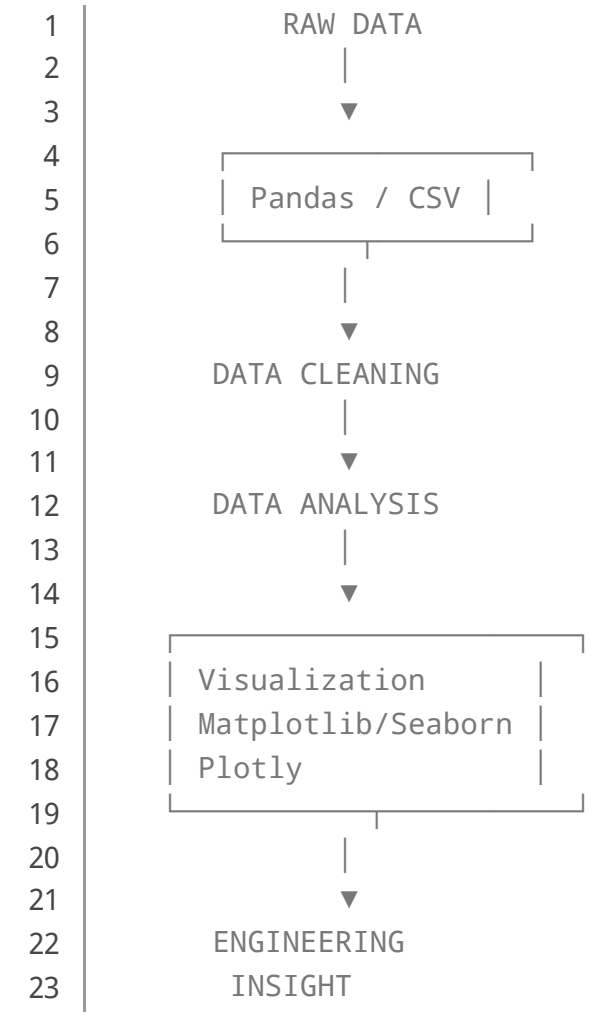
A histogram helps answer:

> *Where are most measurements concentrated?*

This is particularly valuable for quality-control and experimental datasets.

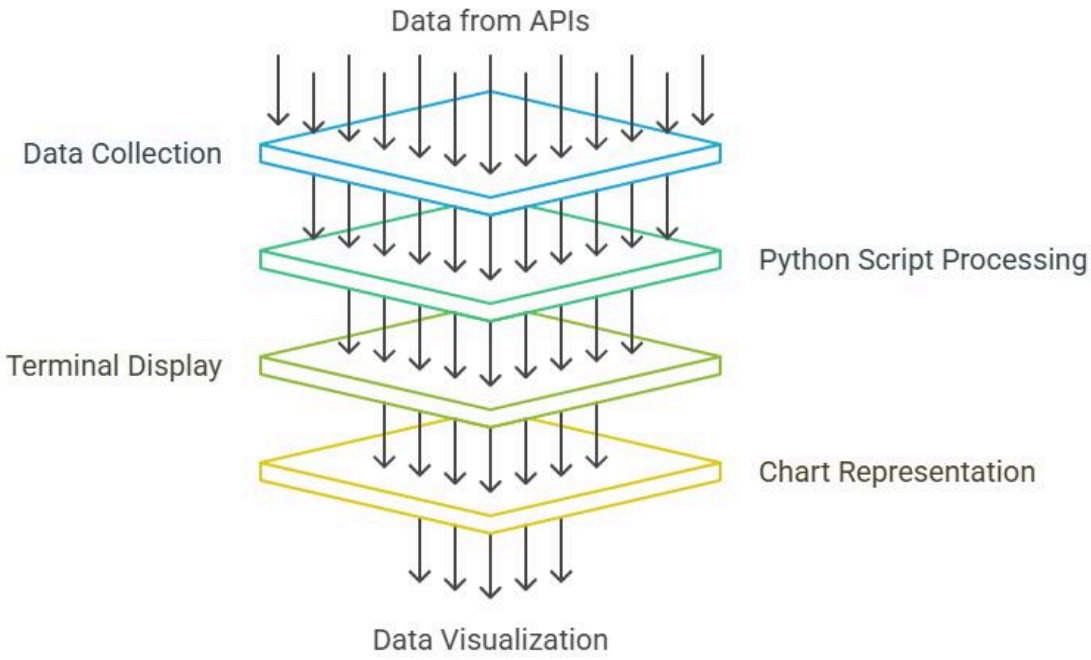
Diagrams and Visualization Architecture

A typical Python visualization workflow can be represented conceptually as:

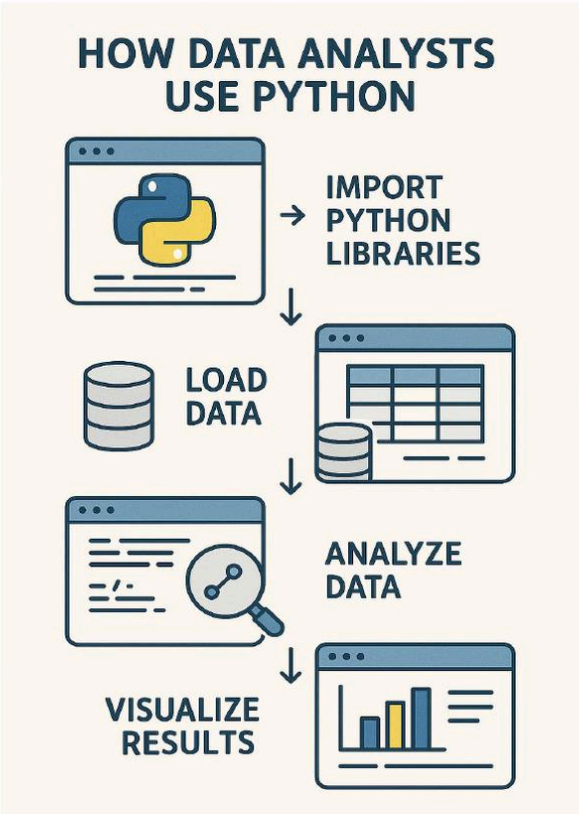


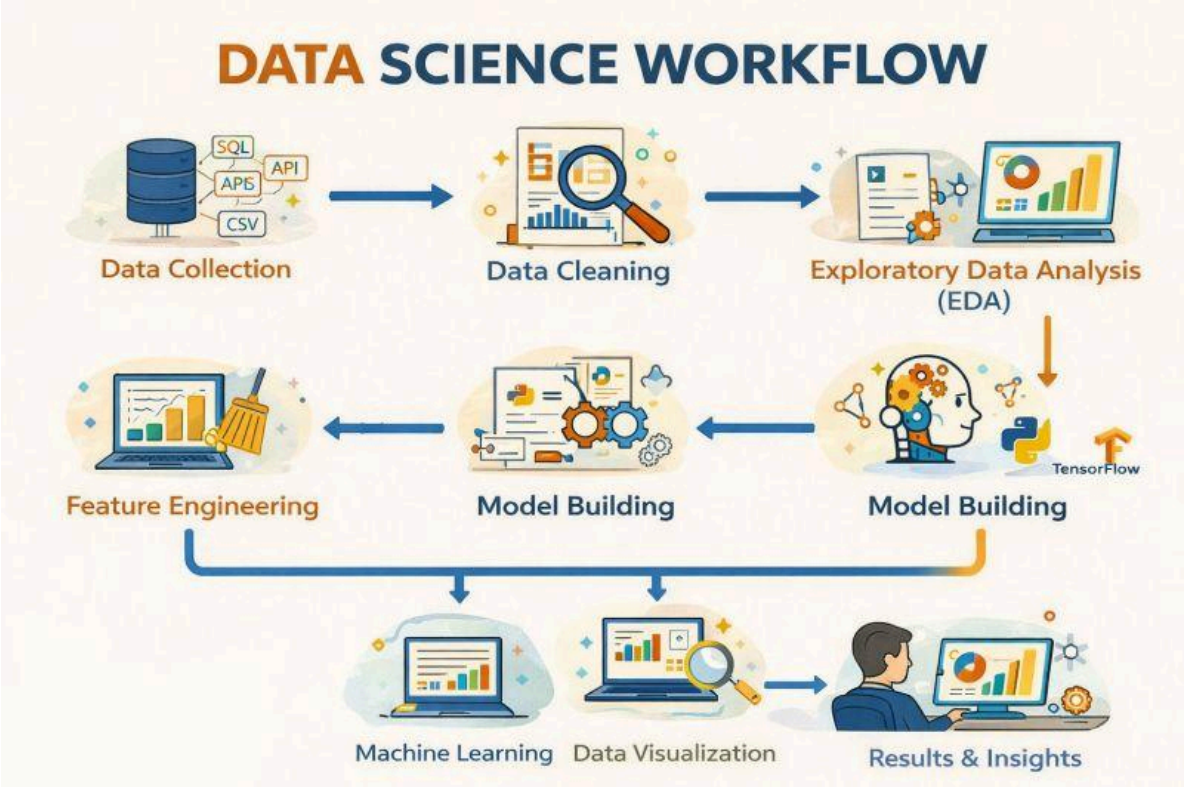
The visualization is therefore only one part of the analytical pipeline.

Data Processing and Visualization Funnel



How Python works in data analysis





A Practical Visualization Decision Tree

You can simplify chart selection:

```
What do you want to understand?
                |
      ┌─────────┼─────────┐
      ▼         ▼         ▼
   Trend    Compare   Relationship
     |         |         |
    Line      Bar     Scatter
     |
     ▼
Distribution?
     |
 Histogram / Box Plot
```

This simple decision process prevents many inappropriate chart choices.

Examples

Example 1: Engineering Temperature Monitoring

Suppose a machine is monitored every minute.

```
import numpy as np
import matplotlib.pyplot as plt

time = np.arange(0, 60, 1)
temperature = 25 + 0.25 * time + np.random.normal(0, 1, 60)

plt.plot(time, temperature)

plt.xlabel("Time (min)")
plt.ylabel("Temperature (°C)")
plt.title("Machine Temperature Monitoring")

plt.grid(True)
plt.show()
```

The graph can reveal whether the machine temperature is:

- Stable
- Increasing
- Oscillating
- Experiencing abnormal spikes

Example 2: Stress-Strain Visualization

A fundamental engineering relationship is stress versus strain:

[\sigma = \frac{F}{A}]

where:

- (\sigma) = stress
- (F) = applied force
- (A) = cross-sectional area

A Python visualization can plot:

```
plt.plot(strain, stress)

plt.xlabel("Strain")
plt.ylabel("Stress (MPa)")
plt.title("Stress-Strain Relationship")

plt.grid(True)
plt.show()
```

The resulting curve can help engineers identify regions such as elastic behaviour and nonlinear response.

Example 3: Correlation Analysis

Seaborn can make exploratory visualization particularly convenient.

```
sns.heatmap(df.corr(numeric_only=True),
            annot=True)

plt.title("Correlation Matrix")
plt.show()
```

A correlation coefficient can be represented as:

[-1 \leq r \leq 1]

where values near (+1) indicate strong positive linear association and values near (-1) indicate strong negative association.

Correlation does **not**, however, automatically establish causation.

Real-World Applications

Mechanical Engineering ⚙️

Visualization can be used for:

- Vibration analysis
- Thermal testing
- Stress-strain experiments
- Engine performance
- Fatigue testing
- Manufacturing quality control

A vibration signal, for example, can be represented as:

[x(t)=A\sin(2\pi ft+\phi)]

Plotting (x(t)) against time can expose periodic behaviour and abnormal oscillations.

Civil Engineering 🏗️

Python visualization can support:

- Structural health monitoring
- Load-deflection analysis
- Concrete testing
- Traffic analysis
- Geotechnical measurements
- Construction progress monitoring

Electrical Engineering ⚡

Engineers can visualize:

- Voltage
- Current
- Power
- Frequency
- Harmonics
- Battery performance

For AC signals:

[v(t)=V_m\sin(\omega t+\phi)]

a graph immediately communicates amplitude, frequency, and phase characteristics.

Energy Engineering 🌱

Visualization can reveal:

- Hourly electricity demand
- Solar generation
- Wind power
- Building energy consumption
- Battery charging/discharging

This allows engineers to identify peak demand and energy inefficiencies.

Common Mistakes

Using the Wrong Chart

A pie chart is not appropriate for every dataset.

If you are investigating a continuous relationship such as:

[Temperature \rightarrow Efficiency]

a scatter or line plot is usually more informative.

Missing Units

Writing:

> *Temperature*

is weaker than:

> *Temperature (°C)*

Engineering charts should communicate measurement units clearly.

Excessive Decoration

Too many colours, labels, annotations, and visual effects can hide the actual information.

A professional engineering chart should prioritize **clarity over decoration**.

Manipulating the Axis

An inappropriate axis range can exaggerate or hide differences.

Always consider whether the axis accurately represents the magnitude of the engineering phenomenon.

Ignoring Missing Data

Before visualization:

```
print(df.isnull().sum())
```

Missing observations can distort averages, trends, and statistical interpretations.

Challenges & Solutions

Large Datasets

Millions of points can make a graph slow and visually overloaded.

Solution: aggregate, sample, or resample the data.

For example:

```
daily = df.resample("D").mean()
```

when working with a properly indexed time series.

Noisy Measurements

Sensors frequently produce noisy signals.

A moving average can provide a simple smoothing technique:

[\bar{x}*t=\frac{1}{N}\sum*{i=0}^{N-1}x_{t-i}]

In Python:

```
df["smooth"] = df["Temperature"].rolling(5).mean()
```

The smoothed curve can then be plotted alongside the original data.

Too Many Variables

Plotting ten variables on one graph often creates confusion.

Solution: use multiple focused charts, faceting, heatmaps, interactive filtering, or carefully designed dashboards.

Case Study: Industrial Motor Monitoring

Imagine an industrial motor operating continuously for several hours.

Sensors collect:

- Temperature
- Current
- Vibration
- Rotational speed

The engineering team wants to determine whether temperature increases before abnormal vibration occurs.

Data Collection

The sensors generate time-series measurements.

Data Processing

Python and Pandas organize the measurements:

```
df = pd.read_csv("motor_data.csv")

df["Time"] = pd.to_datetime(df["Time"])
df = df.sort_values("Time")
```

Visualization

The engineer first creates a temperature trend and then examines vibration.

```
plt.plot(df["Time"], df["Temperature"])
plt.xlabel("Time")
plt.ylabel("Temperature (°C)")
plt.title("Motor Temperature")
plt.xticks(rotation=45)
plt.tight_layout()
plt.show()
```

A second graph can show vibration.

The engineer may discover that vibration increases shortly after temperature begins rising.

That does not automatically prove that temperature caused the vibration. However, it identifies a potentially important relationship for further investigation.

This illustrates the real purpose of visualization:

Raw measurements → visible pattern → engineering hypothesis → deeper analysis → engineering decision.

Essential Tips

Start With the Engineering Question 🎯

Before writing Python code, ask:

> *What am I trying to discover?*

This prevents unnecessary charts.

Keep Charts Simple

Use:

- Clear titles
- Proper units
- Meaningful labels
- Appropriate scales
- Limited visual clutter

Learn Matplotlib First

Matplotlib provides a strong foundation for understanding how Python visualization works.

Once you understand:

```
plt.plot()
plt.scatter()
plt.bar()
plt.hist()
```

you can progress naturally toward Seaborn and Plotly.

Use Seaborn for Statistical Exploration

Seaborn is especially useful when investigating distributions, correlations, categorical data, and statistical relationships.

Use Plotly for Interactivity

Interactive visualization becomes valuable when users need to zoom, hover over points, filter data, or investigate large datasets.

Validate Before Publishing

Never assume that a beautiful graph is a correct graph.

Check:

[\text{Data} \rightarrow \text{Analysis} \rightarrow \text{Visualization}]

Every stage can introduce errors.

FAQs

What is the best Python library for data visualization?

Matplotlib is an excellent starting point because it provides detailed control and is widely used in scientific and engineering applications. Seaborn and Plotly can then extend your capabilities.

Is Python visualization difficult for beginners?

No. Basic charts can be created with only a few lines of code. The more difficult skill is learning how to select and interpret the appropriate visualization.

Should engineers learn Matplotlib or Seaborn first?

Matplotlib is generally a strong foundation because many Python visualization concepts are easier to understand once you know the underlying plotting structure.

Can Python visualize real-time sensor data?

Yes. Python can process and visualize streaming or periodically updated sensor data using appropriate libraries and architectures.

Is Plotly better than Matplotlib?

Neither is universally better. Matplotlib is excellent for scientific and engineering plotting, while Plotly is particularly useful when interactive visualization is required.

Can Python replace Excel for engineering graphs?

Python can replace or complement Excel for many engineering workflows, particularly when datasets are large, repetitive analysis is required, or automation is important.

What mathematics is required?

Basic algebra and statistics are enough to begin. As your work becomes more advanced, concepts such as probability, correlation, regression, Fourier analysis, and numerical methods become increasingly useful.

How long does it take to learn Python visualization?

A beginner can learn basic plotting within a few days. Developing professional visualization skills requires continued practice with real engineering datasets and increasingly complex analytical problems.

Conclusion

Practical Python data visualization is not simply about making graphs—it is about **turning engineering data into understanding**. 📈

A well-designed visualization can expose trends, identify anomalies, compare systems, communicate experimental results, and support better technical decisions.

The fastest learning path is practical:

Load data → clean data → ask a question → choose a chart → visualize → interpret → validate.

Start with Matplotlib, add Pandas for data handling, learn Seaborn for statistical exploration, and introduce Plotly when interactive analysis becomes valuable.

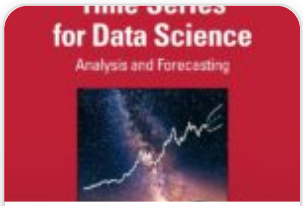
For students, this skill creates a strong bridge between programming, mathematics, and engineering. For professionals, it can transform repetitive analytical tasks into efficient, reproducible workflows.

Ultimately, the most powerful visualization is not the most colourful or complicated one. It is the one that allows an engineer to look at a dataset and quickly understand

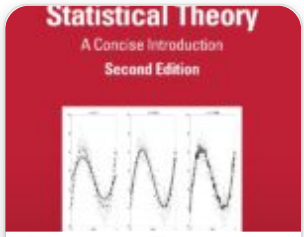
what is happening, why it may be happening, and what should be investigated next. ⚙️📊🚀

Related Posts:

Data Science: Techniques and Intelligent Applications



Time Series for Data Science: Analysis and Forecasting



Statistical Theory 2nd Edition

Power Pivot and Power BI

An Introduction to Statistics: An Active Learning...

Preview PDF Book

← Previous Book

Next Book →

EngiVerse

Explore a vast library of free engineering ebooks curated to help you expand your technical knowledge, conduct groundbreaking research, and advance your career. Our mission is to create a hub of knowledge that fosters innovation, empowers aspiring engineers, and bridges the gap between education and industry.